

Java Backend Architecture

Interview Series

Chapter 19.10

Deployment Strategies & DevSecOps

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 19.10 – Deployment Strategies & DevSecOps

Overview

Modern Java deployments require strategies that minimise risk and downtime: blue-green, canary, rolling, and feature flags decouple deployment from release. DevSecOps integrates security into the CI/CD pipeline (shift left), combining SAST, DAST, SCA, and container signing. Understanding Flagger for automated canary and Cosign for supply chain security is expected at senior level.

Key Definitions

Term	Definition
Blue-Green Deployment	Two identical environments. Switch traffic instantly. Full resource cost; easy rollback.
Canary Release	Route small traffic percentage to new version. Metric-based promotion. Gradual risk exposure.
Rolling Update	Replace pods incrementally. Kubernetes default. Zero downtime; mixed versions during rollout.
Feature Flags	Decouple deploy from release. Toggle features without deployment. A/B testing, kill switches.
Flagger	Kubernetes operator automating canary analysis. Metric-based promotion/rollback.
SAST	Static Application Security Testing. Analyses source code for vulnerabilities (SonarQube, SpotBugs).
DAST	Dynamic Application Security Testing. Tests running application (OWASP ZAP against staging).
SCA	Software Composition Analysis. Scans dependencies for known CVEs (Snyk, Dependabot, OWASP Dependency-Check).
Cosign	CNCF tool for signing container images. Supply chain security: verify image provenance in K8s.
Sigstore	Open-source framework for code signing and verification. Cosign, Fulcio, Rekor components.

Diagram 1: Deployment Strategy Comparison

Strategy	Rollback	Resources	Downtime	Version Mix
Blue-Green	Instant	2x (double) during	None	No (instant cut)
Canary	Delete canary	1x + small canary extra	None	Yes (during ramp)
Rolling	Slow (undo RS)	1x + max surge	None	Yes (during rollout)
Recreate	N/A	1x	Yes	No (down during)

Recommended for Java: Canary (with Flagger) for production risk management

Diagram 2: Flagger Canary Promotion Flow

```

Deploy new version -> Flagger detects Deployment change
|
| 1. Create canary Deployment (weight=5%)
v
Analyse metrics for interval (60s):
- error rate < 1%?
- P99 latency < SLO?
- Custom business metric OK?
|
YES | NO
v   |   v
Increment Rollback
weight   to primary
+10%     (weight=0%)
|
v (repeat until weight=100%)
Promote canary to primary
Delete old primary

```

Code Examples

Flagger Canary for Java K8s Deployment:

```
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: java-api
  namespace: production
spec:
  provider: nginx      # or: istio, linkerd, aws-app-mesh
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: java-api
  service:
    port: 8080
  analysis:
    interval: 1m        # evaluate every minute
    threshold: 5        # fail after 5 consecutive failed checks
    maxWeight: 50       # max canary traffic weight
    stepWeight: 10      # increase by 10% each step
  metrics:
    - name: request-success-rate
      thresholdRange: { min: 99 }    # min 99% success rate
      interval: 1m
    - name: request-duration
      thresholdRange: { max: 500 }   # max 500ms P99
      interval: 1m
  webhooks:
    - name: acceptance-test
      type: pre-rollout
      url: http://test-runner/
      metadata: { type: bash, cmd: "mvn -f acceptance-tests test" }
```

Feature Flags with LaunchDarkly:

```
@Service
public class PaymentService {
    private final LDClient ldClient;

    public PaymentResult processPayment(PaymentRequest request) {
        boolean useNewProvider = ldClient.boolVariation(
            "new-payment-provider",
            new LDContext.Builder(request.getUserId()).build(),
            false // default if flag evaluation fails
        );

        if (useNewProvider) {
            return newPaymentProvider.process(request);
        } else {
            return legacyPaymentProvider.process(request);
        }
    }
}

// Gradual rollout: start at 0%, increase based on metrics
// Kill switch: instantly disable new provider on error spike
// A/B test: measure conversion rate new vs legacy provider
```

DevSecOps Pipeline with Cosign:

```
# GitHub Actions: build, scan, sign
jobs:
  build-sign:
    steps:
      - name: Build Docker image
        run: docker build -t $IMAGE .
```

```
- name: Scan with Trivy (CRITICAL=fail)
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: $IMAGE
    severity: CRITICAL
    exit-code: 1      # fail pipeline on critical CVEs

- name: Sign image with Cosign (keyless Sigstore)
  uses: sigstore/cosign-installer@v3
- run: cosign sign --yes $IMAGE@$DIGEST
  # Signs using OIDC token (no key management)
  # Attestation stored in Rekor (public audit log)
```

```
# GKE Binary Authorization verifies Cosign signature before deployment
```

Interview Q&A – Deployment Strategies & DevSecOps

Q1. When should you use blue-green vs canary deployment?

Blue-green: instant cutover needed (database schema changes requiring all-or-nothing), regulatory requirements for a definitive switch point, simple enough to maintain double resources. Trade-off: double resource cost during transition. Canary: risk reduction via gradual traffic increase, automated metric-based promotion/rollback, production validation with real traffic. Best for Java microservices: canary (with Flagger) provides data-driven deployment safety without sustained double resource cost.

Q2. How does Flagger automate canary analysis?

Flagger watches Kubernetes Deployments. On new image: creates canary Deployment, configures Nginx Ingress/Istio/Linkerd to route canary-weight% to new version. Every interval, evaluates metrics from Prometheus: success rate and P99 latency. If metrics pass threshold for stepWeight intervals: increase weight 10% -> 20% -> 50% -> 100%. If any metric fails threshold threshold times: rollback weight to 0%, restore primary. Zero human intervention for routine deployments.

Q3. What is the difference between SAST, DAST, and SCA?

SAST (Static): analyses source code without executing -- SpotBugs, SonarQube, Semgrep. Finds: SQL injection patterns, insecure random, hardcoded credentials. Fast, runs in CI. DAST (Dynamic): tests running application -- OWASP ZAP, Burp Suite. Finds: authentication bypasses, XSS, actual exploitable vulnerabilities. Slower, runs against staging. SCA (Composition): scans pom.xml for CVE in third-party dependencies -- Snyk, Dependabot. All three complement each other.

Q4. How do feature flags enable zero-downtime Java migrations?

Strangler Fig with feature flags: new Java code deployed alongside old. Flag controls routing: old code for 99% of users, new code for 1%. Gradually increase: 1% -> 10% -> 50% -> 100%. Rollback: flip flag to 0% instantly. Kill switch: instant disable on production incident -- no deployment needed. Dark launch: run new code for all users but hide results (measure performance without exposing new features). Decouples deploy (technical) from release (business).

Q5. What is supply chain security and how does Cosign address it?

Supply chain attack: attacker compromises the path from source to production (CI/CD pipeline, container registry). Cosign signs Docker images with OIDC-based keyless signing (no long-lived keys): signature stored in OCI registry or Rekor (immutable public transparency log). K8s admission controller (Binary Authorization, Kyverno, Sigstore Policy Controller) verifies signature before pod creation. Proves image built by trusted CI/CD pipeline, not tampered after push.

Q6. How do you implement a rolling update in Kubernetes with zero downtime?

Deployment spec: strategy: type: RollingUpdate maxUnavailable: 0 (always maintain full capacity), maxSurge: 1 (add 1 extra pod during update). With PodDisruptionBudget minAvailable: 3: at least 3 pods always available. Java pod lifecycle: preStop hook sleeps 5s (allows LB to de-register), then Spring Boot graceful shutdown drains in-flight requests. terminationGracePeriodSeconds: 30. Prevents dropped requests during rolling update.

Q7. What Snyk/Dependabot rules should trigger CI failure for Java projects?

Fail CI on: CRITICAL CVE in direct dependency (your code directly uses it), HIGH CVE with available fix (no excuse not to upgrade). Warn (not fail) on: HIGH with no fix (fix pending upstream), MEDIUM CVE. Exemptions: document and review regularly. Snyk: --severity-threshold=critical in CI command. GitHub Dependabot: security alerts create PRs automatically. Review weekly. Java projects often have transitive dependency CVEs -- fix by upgrading direct parent dependency.

Q8. How does OWASP Dependency-Check differ from Snyk?

OWASP Dependency-Check: open-source, runs locally, downloads NVD CVE database, analyses JAR file names and hashes to match CVEs. Free, no API key, integrates with Maven plugin. Can have false positives. Snyk: commercial (free tier), uses curated vulnerability database (better accuracy), provides remediation advice, tracks which dependency tree path exposes the CVE, integrates with GitHub/Jira. Both complement each other; Snyk provides better developer experience.

Q9. How do you handle a failed canary deployment?

Flagger automatic rollback: on metric threshold failure, sets canary weight to 0%, alerts via webhook (Slack/PagerDuty). Manual steps: 1) Examine Flagger events: kubectl describe canary java-api. 2) Check canary pod logs for errors. 3) Review Prometheus metrics during canary period. 4) Fix issue in code. 5) Push new image tag -- Flagger detects image change and starts new canary cycle. Never manually patch the canary Deployment (Flagger manages it).

Q10. How do you implement database migrations in a CI/CD pipeline safely?

Flyway/Liquibase: versioned SQL migration scripts in version control, run on app startup or separately in CI. Zero-downtime migrations: backward-compatible changes only (add column, add index, add table -- never drop or rename in same deployment).

Multi-phase: Phase 1 (deploy new code supporting both old and new schema), Phase 2 (migrate data), Phase 3 (remove backward compatibility code). Jenkins/GHA: run migration as Job before deployment, with rollback on failure.